

Worksheet 1 — Security Mindset & Threat Modeling (3 hrs)

Course: Software Security (KOSEN69) · **Week 1 Aligned to:** OWASP 2025 A06 Insecure Design · CWE-501 (Trust Boundary Violation) **Signature game:** "Elevation of Privilege" (Microsoft STRIDE card deck)

Ethics note: This week is *modeling only* — you analyze design, you do **not** attack the app. Run the sample app only on your own VM/localhost. Never apply these techniques to systems you do not own or lack written permission to test.

Part 1 — Student Information

| Name | Student ID | Date | Group | |Andrew Dicesare|6631503050|8/15/2026|---| | | | |

Part 2 — Lecture Questions

Answer in your own words (2–4 sentences each).

1. Define the CIA triad and give one concrete failure example for each of the three properties. = Confidentiality, Integrity, Availability, foundation of software security. 1. Only the right people can read it, 2. No one can silently change it, 3. It's there when you need it.
2. What is a *trust boundary*, and why does data crossing one deserve extra scrutiny? = A trust boundary is a point where data moves between two areas with different levels of trust or control. Data crossing cannot be trusted sometimes.
3. Explain "attack surface." Name two things that increase it in a web app. = attack surface is basically a hole in a system, open ports and large components in a system increase it.
4. What does each STRIDE letter map to, and which security property does each threat violate? = Spoofing - Authentication, Tampering - Integrity, Repudiation - Accountability, Information Disclosure - Confidentiality, Denial of service - Availability, Elevation of privilege - Authorization
5. What does "Secure by Design" (CISA) mean, and how does it differ from bolting security on after release? = CISA means securing the system from the beginning while bolting security is introducing security as a fix or later on.

Part 3 — Hands-on Lab (180 min)

Learning goals: build a data-flow diagram (DFD), apply STRIDE to a real Flask app, rank risks, and propose mitigations. **Prerequisites:** Docker + Docker Compose in your VM; a drawing tool (draw.io / paper + photo); the Elevation of Privilege deck (print or virtual) — free print-and-play PDF at github.com/adamshostack/eop.

Environment setup

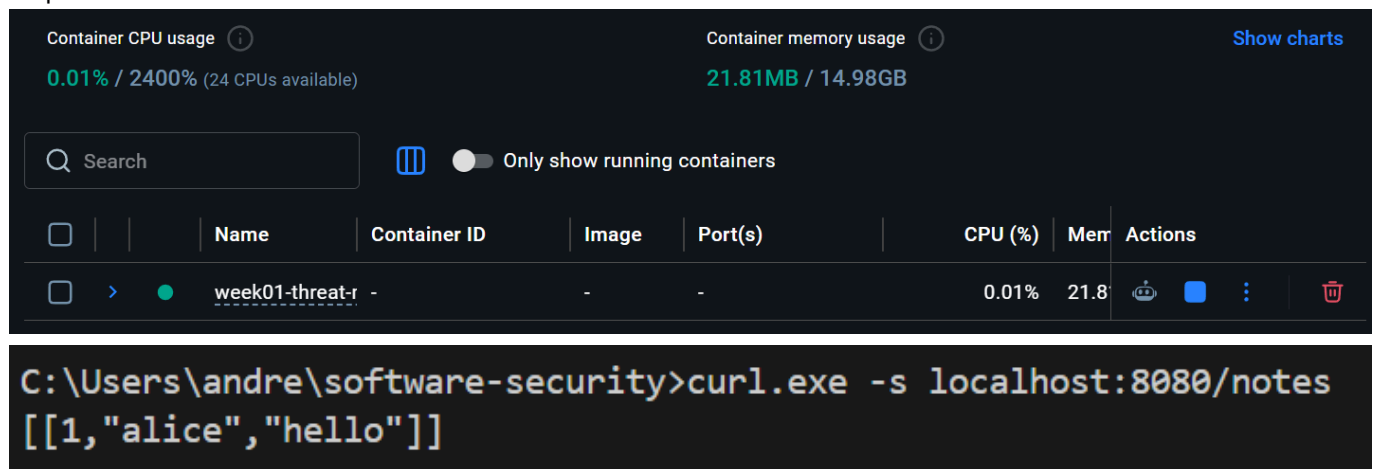
```
cd labs/week01-threat-modeling
docker compose up --build          # starts sample-app on http://localhost:8080
curl -s -X POST localhost:8080/notes -H 'Content-Type: application/json' \
  -d '{"owner":"alice","body":"hello"}' # observe behavior, do not attack
curl -s localhost:8080/notes
```

```
echo "demo file" > demo.txt
curl -s -X POST localhost:8080/upload -F "file=@demo.txt" # observe behavior, do not attack
curl -s localhost:8080/files/demo.txt
```

Source to model lives in `sample-app/app.py`. Template to fill: `THREAT-MODEL-TEMPLATE.md` (copy it, do not edit the original).

What to submit per task: the threat/element identified + a screenshot (DFD, table, or running app) + a 2–3 sentence mitigation.

Task 0 — Onboarding (5 min) · *Goal:* prove the environment works. *Steps:* `docker compose up`, hit `/notes` and `/files/<name>`, read `sample-app/app.py`. *Deliverable:* screenshot of the running app + the JSON response.



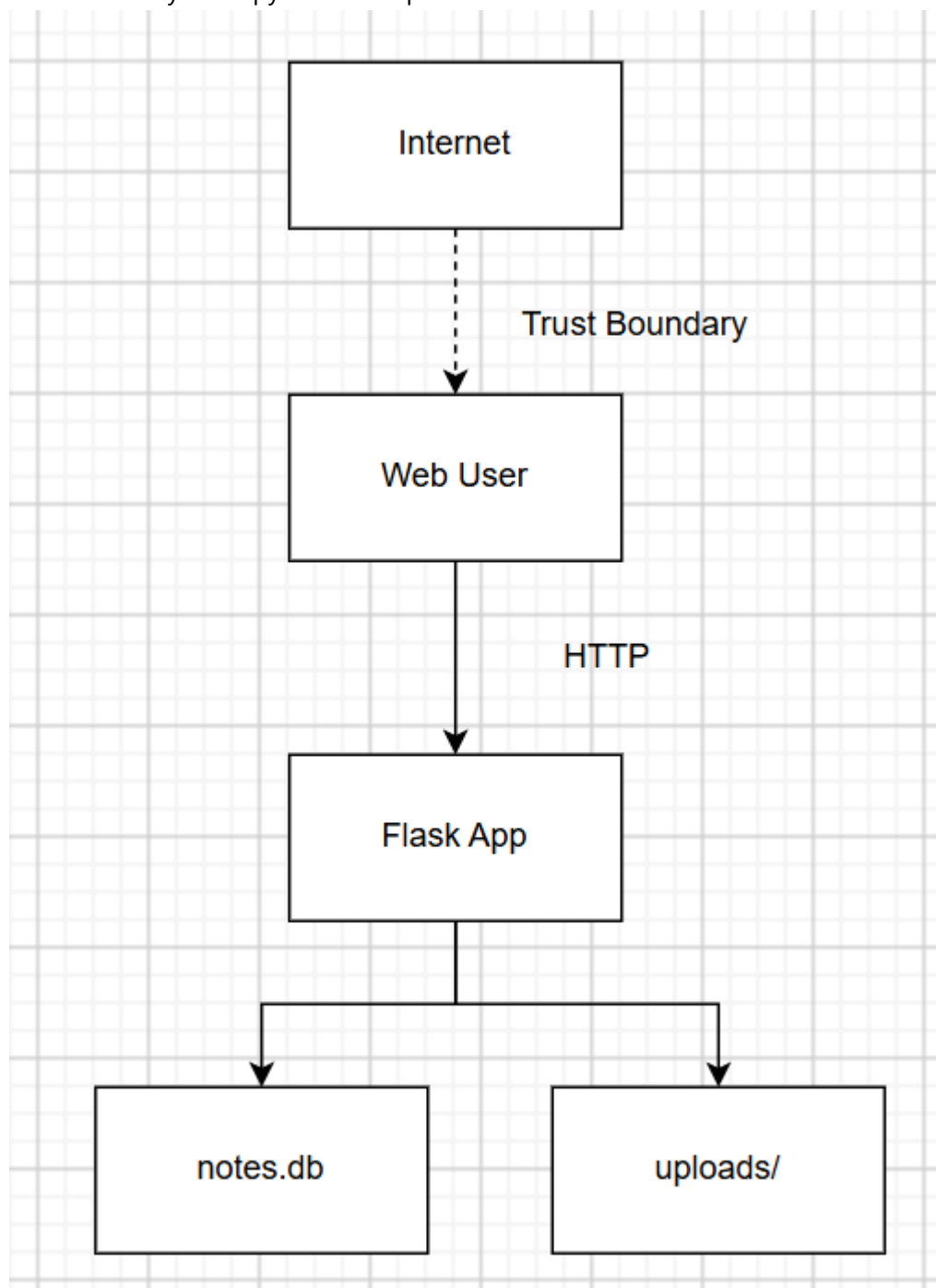
The screenshot shows Docker Desktop's container management interface. At the top, it displays 'Container CPU usage' as 0.01% / 2400% (24 CPUs available) and 'Container memory usage' as 21.81MB / 14.98GB. Below this is a search bar and a toggle for 'Only show running containers'. A table lists the containers, with one container named 'week01-threat-r' visible. Below the Docker interface, a terminal window shows the command `C:\Users\andre\software-security>curl.exe -s localhost:8080/notes` and its output `[[1,"alice","hello"]]`.

	Name	Container ID	Image	Port(s)	CPU (%)	Mem	Actions
☐	week01-threat-r	-	-	-	0.01%	21.8	

```
C:\Users\andre\software-security>curl.exe -s localhost:8080/notes
[[1,"alice","hello"]]
```

Task 1 — Draw the DFD (25 min) · *Goal:* map the system. *Steps:* identify the external entity (web client), the process (Flask app), the data store (`notes.db` SQLite), the `uploads/` store, and the flows for `/notes`, `/upload`, `/files/<name>`; mark the Internet→app trust boundary with a dashed line. *Deliverable:* DFD image

embedded in your copy of the template.

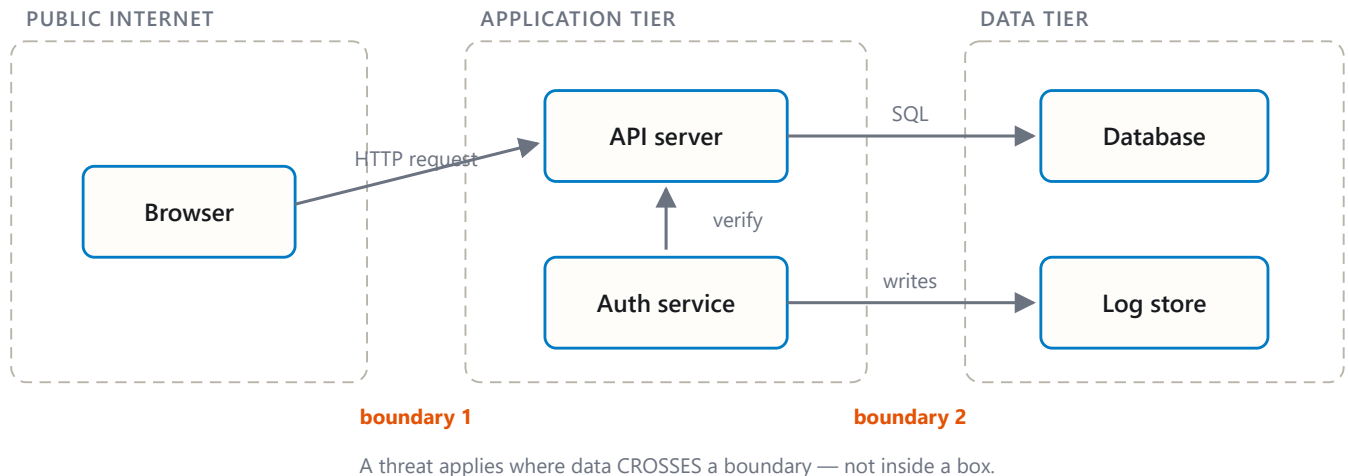


Task 2 — STRIDE the elements (30 min) · *Goal:* enumerate threats per element. *Steps:* for each element fill the S/T/R/I/D/E grid. Ground it in real code: `/notes` accepts a client-supplied `owner` with no auth (Spoofing); `/upload` saves raw `f.filename` — arbitrary-file-write (Tampering) — and echoes the resolved save path back in its response (Information disclosure); `/files/<name>` reads it back but is comparatively defended (see Task 5); no logging anywhere (Repudiation). *Deliverable:* completed STRIDE table.

Task 3 — Elevation of Privilege game (20 min) · *Goal:* find threats you missed. *Steps:* play the EoP deck against your DFD; each card you can tie to a real element/flow scores a point; record every valid threat. No printer or scissors? Draw from the digital deck below instead — same 78 cards, same rule. *Deliverable:* list of carded threats + score.

eop-deck

Task 3b — Systems-level pass (25 min) 🕒 · *Goal:* find what the per-element grid cannot see. Tasks 2 and 3 enumerate threats **one element at a time**, and that is exactly where threat models are known to stop short — students taught STRIDE alone reliably identify component threats and *discount system-level ones* (Joshi et al., ASEE 2024). So do a second pass over the **whole** diagram:



- **Trust boundaries end-to-end.** Follow one request from the client to `notes.db` and back. List every boundary it crosses. Which crossing has no check on it?
- **Assume one element is fully owned.** Pick the Flask process, then the `uploads/` store. For each: what does the attacker now *reach* — not what is it, but where does it get them?
- **Chain two "low" findings.** Find two threats you or the EoP deck rated minor that combine into something you would not accept. Write the chain as **A** → **B** → **consequence**.
- **One-line system claim.** Finish: "Even if every element-level mitigation in Task 8 is implemented, this system still fails if ____."

Use the simulation below before you start — toggle a component to attacker-controlled and watch what it reaches:

trust-boundary

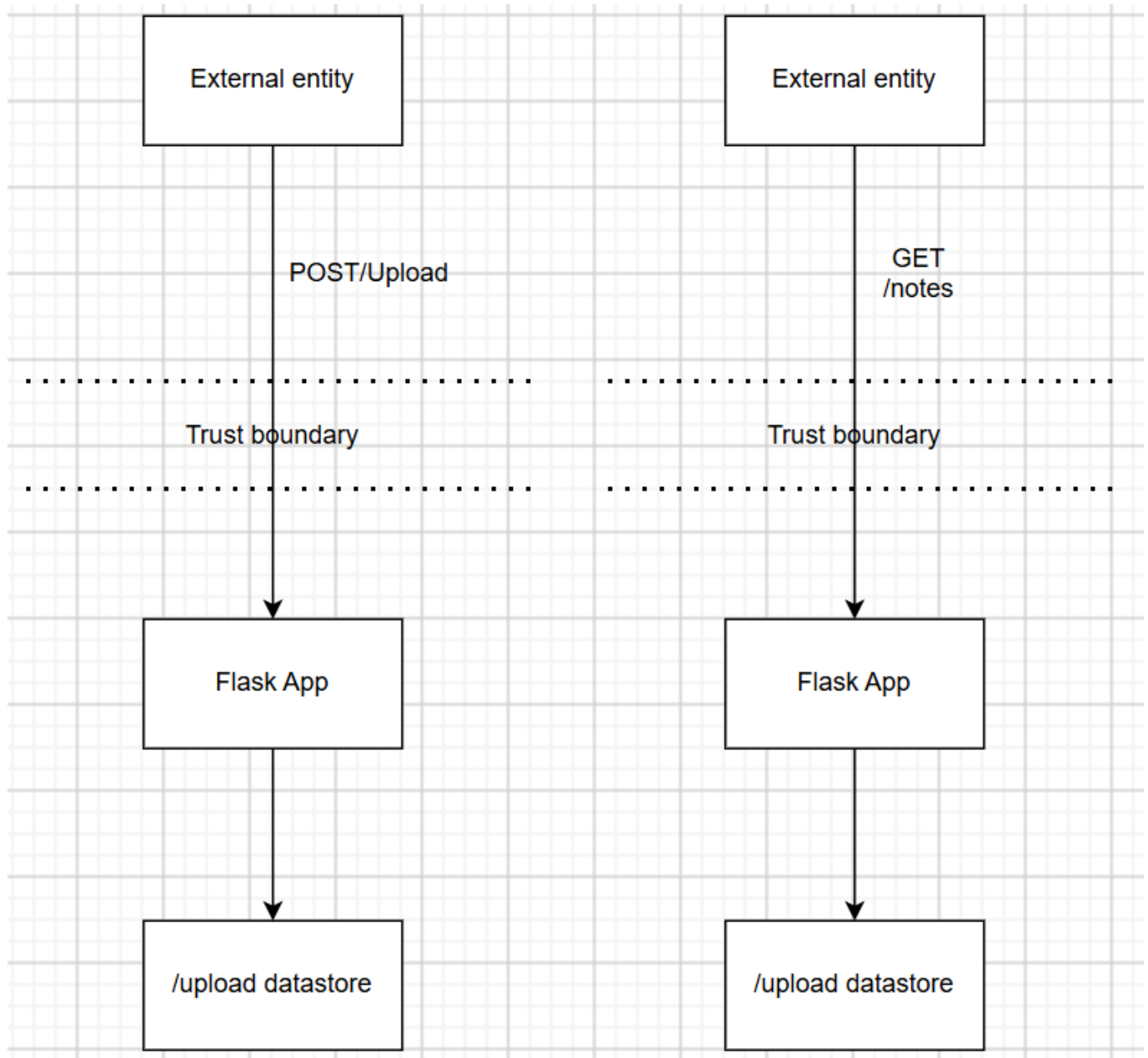
Deliverable: the boundary list, two owned-element reachability notes, one written chain, and the system claim.

Task 4 — Abuse cases & attacker personas (20 min) · *Goal:* think like specific adversaries. *Steps:* define 2 personas (e.g. a curious logged-in user; an anonymous internet attacker) and write 2 abuse cases each against the sample app, tied to DFD elements. *Deliverable:* 4 abuse cases.

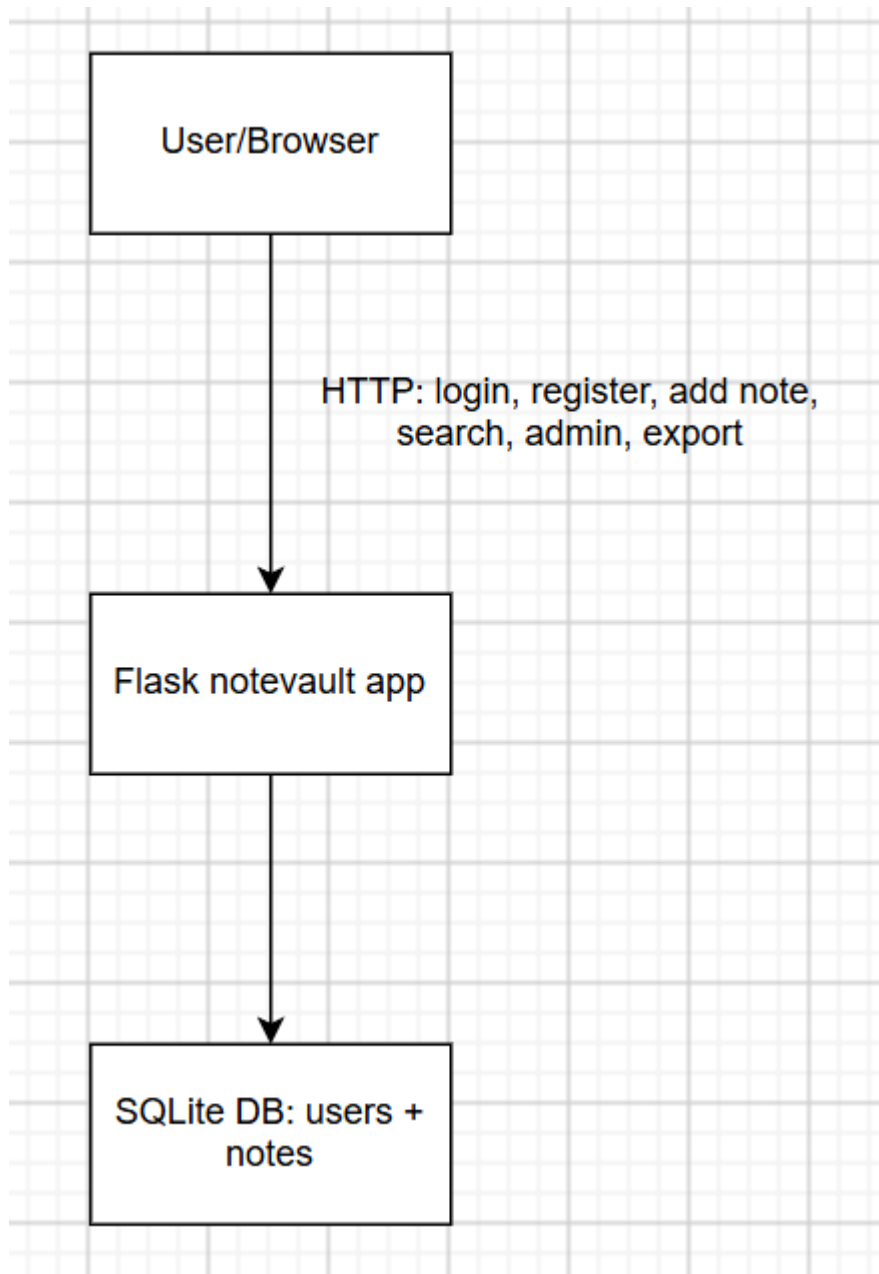
Persona 1: Anonymous attacker - Abuse case 1: the attacker accesses the notes and breaches confidentiality. Abuse case 2: the attacker writes a script to upload large 5gb junk files to the `/upload` endpoint which causes a denial of service.

Persona 2: Malicious Insider - Abuse case 1: the insider is able to upload notes using the other person's name which is spoofing and attacking the integrity Abuse case 2: The insider uploads a script or file that analyzes the http response and echoes the response on where the file was saved which can be leaked attacking the Information Disclosure on /upload

Task 5 — Path-traversal deep-dive (25 min) · Goal: analyze the riskiest flow. Steps: trace `/upload` → `/files/<name>`; explain how `../` in a filename escapes `uploads/`; sketch the secure design (`secure_filename`, store outside web root, allow-list extensions). Deliverable: the data flow + secure-design note.



Task 6 — Threat-model the project target (30 min) · Goal: kick off your term project. Steps: stop the sample-app first (`docker compose down` — both apps bind host port 8080), then run **NoteVault** (`cd ../../project/starter-app && docker compose up`), draw a quick DFD, and list the top 3 STRIDE threats you'd investigate. Deliverable: NoteVault DFD + top-3 threats (reuse these in your project report — `project/REPORT-TEMPLATE.md` in the repo root).



1 Spoofing 2 SQL injection 3

Command injection

Task 7 — Security requirements (15 min) · *Goal:* turn threats into testable requirements. *Steps:* write 3 security requirements as acceptance criteria ("the system must ... so that ..."), each mapped to a threat from Task 2 or Task 6. *Deliverable:* 3 testable security requirements. 1 The system must require a valid authenticated session for all protected routes, so that an attacker cannot spoof another user. 2 The system must use parameterized queries for all database access, so that user input cannot trigger SQL injection. 3 The system must reject untrusted export input and never execute shell commands with user-controlled data, so that attacker input cannot run OS commands.

Task 8 — Defend / fix it: rank & mitigate (25 min) · *Goal:* turn threats into action you can prove. *Steps:* rank the top 5 threats by likelihood × impact; propose one concrete mitigation each (e.g., auth on `/notes`, `secure_filename()` + allowlist for `/upload`, request logging for Repudiation, size/rate limits for DoS). Then **pick one and actually implement it** in your fork. Answer:

1 SQL injection in login/search 2 Command injection in export 3 Forged session / auth bypass 4 Admin data exposure 5 Missing authorization checks on protected actions

picked SQL injection implemented the SQL injection fix in app.py by replacing unsafe string-built SQL in the login and search routes with parameterized queries.

Deliverable — the top-5 table, plus for the one you implemented:

1. the **diff** (commit hash on your `wk01` branch), HASH: `cfbdd4bc9bc77308e33b386327b93b4302f23b45`
2. **evidence it works**: the request that succeeded before your change and is refused after — both outputs, Before: a login request with `admin' OR '1'='1` succeeded and redirected to `/` After: the same request returned 401 login failed
3. **why it closes the class, not the instance** (2–3 sentences). `secure_filename()` on one endpoint is an instance fix; *"no user-supplied string ever becomes a path component"* is a class fix. Say which yours is, and if it's an instance fix, say what the class fix would be. This is a class fix because the root cause was untrusted input being interpreted as SQL, not just one bad login string. The class-level fix is to use parameterized queries for all database access so user input is never executable code.

Why this is weighted. Fewer than half of working developers can spot a security hole in code, and being shown vulnerabilities does not by itself teach you to find or close them. Exploiting is the half that feels like progress; defending is the half that transfers to your job.

Part 4 — Reflection

1. Map your top finding to a CWE and to OWASP A06 (Insecure Design); explain the mapping in one sentence. Answer: My top finding is SQL injection, which maps to CWE-89 and OWASP A06. The design flaw was that user input was allowed to influence SQL logic without proper validation or parameterization.
2. Name one real-world breach caused by a design flaw (not a missing patch) and what design control would have prevented it. Answer: One real-world example is the Equifax breach, caused by an unpatched Struts vulnerability. A stronger design control, such as secure-by-default validation and a disciplined patching process, could have prevented it.
3. Of your five mitigations, which gives the most risk reduction per unit of effort, and why? Answer: Parameterized queries give the biggest risk reduction per unit of effort because they stop a whole class of injection bugs with one standard fix and apply across all database interactions.

Grading rubric (100)

Criterion	Points
Lecture questions (Part 2)	20
Exploitation + evidence (DFD + STRIDE table + EoP findings + screenshots)	40
Defense (top-5 ranking + mitigations)	25
Reflection (CWE/OWASP mapping + breach + best mitigation)	15

Assessed within the rows above (they are not extra points — they are what those points are for):

- **Systems-level reasoning** (inside *Exploitation + evidence*, Task 3b): does the model reach past single elements to boundaries, reachability and chains? Scored with the STRIDE + systems-thinking rubrics of

Joshi et al. 2024.

- **Defensive proof** (inside *Defense*, Task 8): a claimed mitigation with no before/after evidence scores at most half. A mitigation you can show closing a *class* scores full.
- **Adversarial thinking** (across the whole sheet): do the abuse cases, personas and chains show you reasoning as an attacker with goals and constraints — or just listing categories? This is the course's central disposition and it is assessed, not assumed.

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` in the same image as the evidence.

When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a

```
C:\Users\andre\software-security>echo %USERNAME% 6631503050 %date% %time%
andre 6631503050 Sun 08/16/2026 8:02:56.31
```

cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.

- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit: learn.zcr.ai/submit (full guide: [SUBMISSION.md](#) in the repo root).*
- **Explain in your own words** (*graded on your reasoning, not copied text*):
 1. What did you do, and **why did the vulnerability work**? Answer: I looked at the Flask app and found that the login and search endpoints built SQL queries by concatenating user input directly into the query string. This worked because the database treated the injected ' OR '1'='1 text as part of the SQL logic, so it bypassed the normal username/password check and returned matches it should not have.
 2. **Why does your fix actually stop it** — and what could still break it? Answer: My fix replaced those unsafe query strings with parameterized queries, so the user input is treated as data instead of executable SQL. This stops the injection because the database never interprets the attacker's input as SQL syntax. What could still break it is other routes or future code that reintroduces string-built SQL, so the real fix is to use parameterized queries consistently everywhere.



Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

I asked an AI to fix the SQL injection issue and it suggested a parameterized query, but the answer was incomplete because it only focused on one route and did not mention that the same vulnerability existed in the search query too. The risky part was treating the fix as one-off instead of a class-level fix.

Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

This app takes user input and puts it straight into an SQL statement. Because the input is not sanitized or bound properly, an attacker can send a value like ' OR '1'='1 and change the logic of the query, which lets them bypass login or leak data.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Fix the SQL injection vulnerability in this Flask app. The login and search routes currently build SQL queries by concatenating user input. Replace all unsafe string-built SQL with parameterized queries using bound values, and explain why the fix prevents injection while preserving valid behavior. Verify the fix by testing a payload like admin' OR '1'='1 against the login route and confirming it fails with a 401.

Verified result:

The exploit no longer succeeds. The login attempt with the injection payload is rejected with 401 login failed. The fix is implemented in app.py.